LECTURES

05 ES6 classes | 05 Inheritance, abstract, static | 06 Private, protected, modules | 06 Generators, iterators | ★ Cheat sheet

REVISION NOTES · JAVASCRIPT

JavaScript Classes and ES6 Notes

Condensed from two decks: ES6 Object and ES private, protected, member. Code is cleaned up from the slides (curly quotes and typos fixed). Points marked "Extra" go slightly beyond the slides.

ES6 classes

Deck 05 · ES6Object

ES6 introduced the `class` keyword. It replaces the constructor-function and prototype style from earlier lectures.

```
'use strict'
class Account {
  constructor(name, amt){
    this.name = name;
    this.balance = amt;
    alert(typeof this.constructor);    // "function"
  }
}
var obj1 = new Account("raj", 20000);
obj1 instanceof Account;    // true
obj1 instanceof Object;     // true
obj1.name = "geeta";        // members are public; changes the value
document.write(obj1);       // [object Object]
```

- `constructor` runs on `new` and creates the instance members with `this`.
- A class is really a function (`typeof this.constructor` is `"function"`), but you **cannot call it without `new`**. `Account()` gives an error.
- Printing an object calls `toString()`. If the class has none, the parent `Object`'s version runs and gives `[object Object]`.

What the class keyword changes

RULE	DETAIL

Methods are non-enumerable	The class sets the enumerable flag to false for all methods on the prototype, so `for...in` does not list them. That is what you want.
Always strict mode	All code inside the class is automatically in strict mode.
Not hoisted	Function declarations are hoisted; class declarations are not. Using the class before its declaration gives `ReferenceError`.

```
const p = new Rectangle();   // ReferenceError
class Rectangle {}
```

Enumerable: the difference you can see

Members added with `this.x = ...` inside a constructor function are enumerable, so `for...in` lists them, including a method like `dojob`. Methods written in a class body are not listed.

```
let person = function(nm, ag){
  this.name = nm;
  this.age = ag;
  this.dojob = function(){ document.write("do job hello<br/>"); };
};
let obj = new person("Raj", 25);
for (let prop in obj){
  if (typeof obj[prop] == 'function') obj[prop]();   // dojob shows up here
  document.write(prop + " " + obj[prop]);
}
```

Class expressions

A class can be used as a value: assigned, passed around, returned, or created on demand.

```
let User = class {
  sayHi(){ alert("Hello"); }
};
new User().sayHi();

function makeClass(phrase){
  return class { sayHi(){ alert(phrase); } };   // returns a new class each time
}
let User2 = makeClass("Hello");
new User2().sayHi();                            // Hello
```

Methods and toString

Methods and toString

```
class Account {
  constructor(name, amt){ this.name = name; this.balance = amt; }
  deposit(amt){ this.balance += amt; }
  toString(){ return this.name + "  " + this.balance; }   // overrides Object's toStri
}
var obj1 = new Account("raj", 20000);
document.write(obj1);          // raj  20000
obj1.deposit(500);
```

Methods live once on the class prototype, not inside each object. Each object holds only its own data (`name` , `balance`). A method calls another with `this.` , for example `cube()` uses `this.sqr()` .

Getters and setters

Using `get` and `set` keeps validation in one place while callers still write plain property syntax.

```
class Account {
  constructor(name, amt){
    this.Aname = name;        // (1) calls the SETTER, not a field
    this.Abalance = amt;
  }
  get Aname(){ return this._name; }
  set Aname(value){ this._name = value; }        // stores in the real field _name
  get Abalance(){ return this._balance; }
  set Abalance(value){ this._balance = value; }
  deposit(amt){ this.Abalance = this.Abalance + amt; }
  toString(){ return this.Aname + "  " + this.Abalance; }   // calls the getters
}
var obj1 = new Account("raj", 20000);
obj1.Aname = "geeta";          // calls set
obj1.Aname;                    // calls get
```

- The getter and setter need a **different name** from the stored field (`Aname` vs `_name`), otherwise the setter calls itself forever.
- The object then holds `_name` and `_balance` . The accessors sit on the prototype, so `obj.hasOwnProperty("Aname")` is `false` and `obj.hasOwnProperty("_name")` is `true` .

Inheritance, abstract classes, static members

Deck 05 · ES6Object

extends and super

```
class Account {
  constructor(fname, lname, amt){
    this.firstname = fname;
    this.lastname = lname;
    this.balance = amt;
  }
  set firstname(value){ this._fnm = value; }
  get firstname(){ return this._fnm; }
  set balance(value){ this._bal = value; }
  get balance(){ return this._bal; }
  set lastname(value){ this._lnm = value; }
  get lastname(){ return this._lnm; }
  deposit(amt){ this.balance += amt; }
  toString(){ return this.firstname + "  " + this.lastname + " " + this.balance; }
}

class Savingaccount extends Account {
  constructor(acctype, fname, lname, amt){
    super(fname, lname, amt);       // runs the parent constructor
    this.type = acctype;
  }
  withdraw(amt){
    const minbal = 1000;
    if (amt > (this.balance - minbal)) throw new TypeError("err");
    this.balance -= amt;
  }
}

var obj = new Savingaccount("sav", "Raj", "Mathur", 2000);
obj.deposit(2000);              // balance 4000 (parent method)
try { obj.withdraw(5000); }     // would leave less than 1000, so it throws
catch(e){ console.log(e.message); }   // "err"
```

- `extends` makes Account the parent and Savingaccount the child. The child gets the parent's methods (`deposit` , `toString`) and adds its own (`withdraw`).
- `super(...)` calls the parent constructor. In a child constructor it must come **before** any use of `this` (Extra).
- This replaces the older `Parent.call(this, ...)` and `Object.create(Parent.prototype)` pattern.
- `throw new TypeError("err")` raises an error; `try / catch (e)` handles it and `e.message` gives the text.

Abstract class with `new.target`

JavaScript has no `abstract` keyword. Inside a constructor, `new.target` is the class that `new` was actually used on. If it is the base class itself, throw.

```
class Account {
  constructor(fname, lname, amt){
    if (new.target === Account)
      throw new TypeError("Cannot construct Abstract instances directly");
    this.firstname = fname; this.lastname = lname; this.balance = amt;
  }
}
class Savingaccount extends Account { /* ... */ }

var a = new Account("Raj", "Mathur", 2000);          // TypeError
var s = new Savingaccount("sav", "Raj", "Mathur", 2000);   // works: new.target is Sav
```

Static members

A `static` member belongs to the class itself, not to its objects. Call it with the class name. It lives once in memory, outside any object.

```
class mymaths {
  static add(a, b){ return a + b; }
  static sqr(a){ return a * a; }
}
mymaths.add(3, 5);    // 8
mymaths.sqr(3);       // 9
```

```
class Account {
  static accid(){                    // auto-increment id
    Account.dd = ++Account.dd || 1;  // first time dd is undefined, ++ gives NaN, so |
    return Account.dd;
  }
  constructor(name, amt){
    if (new.target === Account) throw new TypeError("Cannot construct Abstract instanc
    this.Aname = name; this.Abalance = amt;
    this._id = Account.accid();      // every new account gets the next id
  }
  get Aid(){ return this._id; }      // getter only: id can be read, not set
}
```

bind, call and this

A method pulled out of its object loses its `this`. `bind` fixes `this` permanently and returns a new function; `call` runs the function once with a chosen `this`.

```
const module = { x: 42, getX: function(){ return this.x; } };

const unboundGetX = module.getX;
unboundGetX();                          // undefined  (this is the global window)

const boundGetX = unboundGetX.bind(module);
boundGetX();                            // 42

module.getX.call(module);               // 42
```

Extra: in strict mode the unbound call throws a TypeError instead, because `this` is `undefined`.

Where `document.write` comes from

`document.write("hello")` is short for `window.document.write("hello")`. `window` is the global object, `document` is a member of it, and `write`, `alert`, `open` are methods. Under `'use strict'` a typo such as `documet.write` is an error.

Private, protected and static members; modules

`Deck 06 · ES_priv_pro_member`

Private fields: `#`

A name starting with `#` is private to the class. This gives real data hiding (encapsulation). The slides label this feature "ES8"; Extra: it is officially ES2022.

```
'use strict'
class Account {
  #name;                        // must be DECLARED in the class
  #balance;
  constructor(name, amt){
    this.#name = name;
    this.#balance = amt;
  }
  toString(){ return `Name=${this.#name} Bal=${this.#balance}`; }   // template litera
}
var obj1 = new Account("raj", 20000);
document.write(obj1);          // Name=raj Bal=20000  (toString called)
obj1.#name = "geeta";          // SyntaxError: Private field '#name' must be declared
```

- Private fields **must be declared** at the top of the class. Declared but unassigned, the default value is `undefined`.
- They cannot be read or written outside the class. The error is a SyntaxError at load time

- They cannot be read or written outside the class. The error is a SyntaxError at load time, not a runtime message.
- Calling `Account()` without `new` is still an error.
- Template literals use backticks and `${...}` to put values into a string.

Protected by convention: _

JavaScript has no real `protected`. A leading underscore is only a reminder.

```
class Account {
  _name;
  _balance;                    // declaring the field is optional here
  constructor(name, amt){ this._name = name; this._balance = amt; }
}
var obj1 = new Account("raj", 20000);
obj1._name = "geeta";          // works, nothing stops it
```

	`#name` **PRIVATE**	`_name` **"PROTECTED"**
Enforced by the language	Yes	No, only a naming habit
Access from outside the class	Error	Possible, but you should not
Access from a child class	No (see note below)	Intended: use it in child classes only
Must declare the field in the class	**Yes**	No

Design from the deck: private data, protected accessors

```
class Account {
  #firstname; #lastname; #balance; #id;
  static #aid = 0;                          // private static: shared id counter
  constructor(fname, lname, amt){
    this.firstname = fname;                 // goes through the setters
    this.lastname = lname;
    this._cbalance = amt;
    this.#id = ++Account.#aid;              // access static with the class name
  }
  set firstname(value){ this.#firstname = value; }
  get firstname(){ return this.#firstname; }
  set _cbalance(value){ this.#balance = value; }   // "protected" accessor
  get _cbalance(){ return this.#balance; }
  get gid(){ return this.#id; }             // getter only: read-only id
  deposit(amt){ this._cbalance += amt; }
  toString(){ return this.gid + " " + this.firstname + " " + this.lastname + " " + thi
}
class Savingaccount extends Account {
```

```
  type;
  constructor(acctype, fname, lname, amt){ super(fname, lname, amt); this.type = accty
  withdraw(amt){
    const minbal = 1000;
    if (amt > (this._cbalance - minbal)) throw new TypeError("err");
    this._cbalance = this._cbalance - amt;
  }
}
```

- **Read-only id:** only a getter (`get gid`), no setter. Users can read an account number but cannot change it, because the bank assigns it. Trying `obj.gid = 101` is not allowed.
- **Auto-increment:** `static #aid` is shared by all accounts. `++Account.#aid` gives each new object the next id.
- **Why a getter and a setter for balance:** you must be able to change it when money is deposited, so a setter is needed. The name `_cbalance` signals "use inside the class and child classes only".
- **Why a child class cannot touch `#balance` :** private means that class only. The child goes through the accessor `_cbalance` that the parent exposes.
- In memory: the static values (`aid` , company name, rate of interest, `payinterest()`) exist once, outside the objects. Each object holds its own fields (name, balance, id, type).

Private methods and accessors

```
class ClassWithPrivateMethod {
  #privateMethod(){ return 'hello world'; }
  getPrivateMessage(){ return this.#privateMethod(); }
}
new ClassWithPrivateMethod().getPrivateMessage();    // "hello world"

class ClassWithPrivateAccessor {
  #message;
  get #decoratedMessage(){ return `${this.#message}`; }
  set #decoratedMessage(msg){ this.#message = msg; }
  constructor(){
    this.#decoratedMessage = 'hello world';
    console.log(this.#decoratedMessage);
  }
}
```

Private instance methods are restricted in the same way as private fields: available on instances, usable only inside the class.

Private static fields and methods

Private static fields and methods

```
class ClassWithPrivateStaticField {
  static #PRIVATE_STATIC_FIELD;
  static publicStaticMethod(){
    ClassWithPrivateStaticField.#PRIVATE_STATIC_FIELD = 42;
    return ClassWithPrivateStaticField.#PRIVATE_STATIC_FIELD;
  }
}
ClassWithPrivateStaticField.publicStaticMethod() === 42;   // true

class ClassWithPrivateStaticMethod {
  static #privateStaticMethod(){ return 42; }
  static publicStaticMethod1(){ return ClassWithPrivateStaticMethod.#privateStaticMeth
  static publicStaticMethod2(){ return this.#privateStaticMethod(); }
}
```

- Static variables can be used only by static methods.
- Private static members are called on the class itself and are accessible only inside the class declaration. Private static methods can be generator, async, or async generator functions.

PROVENANCE RESTRICTION

Only the class that defines a private static member can access it. Using `this` in a static method is risky, because in a subclass call `this` is the **SUBCLASS**.

```
class Base {
  static #privateStaticMethod(){ return 42; }
  static publicStaticMethod1(){ return Base.#privateStaticMethod(); }   // uses th
  static publicStaticMethod2(){ return this.#privateStaticMethod(); }   // uses th
}
class Derived extends Base {}
Derived.publicStaticMethod1();   // 42
Derived.publicStaticMethod2();   // TypeError, this is Derived, not Base
```

Rule: inside static code, refer to private static members by the **CLASS NAME**.

Modules: export and import

Put each class in its own file and share it with `export` and `import`. The deck saves the files with the `.mjs` extension and runs them with Node.

```
// Account.mjs
export class Account { /* ... same class as above ... */ }

// Saving.mjs
import { Account } from "./Account.mjs";
export class Savingaccount extends Account { /* ... */ }

// use.mjs
import { Savingaccount } from "./Saving.mjs";
const sobj = new Savingaccount("sav", "raj", "Mathur", 5000);
console.log(sobj.toString());
sobj.withdraw(1000);
console.log(sobj);

// run:  node use.mjs
```

- `export` makes a class available outside its file. `import { Name } from "path"` brings it in; the braces hold the exported name.
- The first slide of this deck sketches the idea of a namespace: `Vita.DAC` is class `DAC` inside `Vita`, and a module import plays that role.

Practice tasks from the deck

BANK

Abstract class `Account` with private id, name, balance.
Name getter/setter, length 4 to 15.
Id auto-increment, getter only.
Balance getter/setter "protected".
`deposit()` increases balance.
Child classes Saving and Current with `withdraw()` : saving must not go negative, current may.
Make 2 objects, print name and balance with `toString`.
Print the bank name when the application loads.
Static `payinterest(accountholder)` adds interest. Static `interest = 0.06`.

EMPLOYEE

Abstract class `Employee` with private id, name, salary, netsalary.
Name getter/setter, length 4 to 15.
Id auto-increment, getter only.
Salary and netsalary getters/setters "protected".
`calculate_netsalary()` deducts TDS and sets netsalary.
Child classes permanentemp and contractemp with `givebonus()` : 10% and 5%.
Make 2 objects, print name and netsalary with `toString`.

Make 2 objects, print name and netsalary with 2 decimals.

Print the company name when the application loads.

Static `TDS = 0.1` .

```
// Skeleton for the Bank task
class Account {
  #id; #name; #balance;
  static #aid = 0;
  static interest = 0.06;
  static { console.log("Vidya Bank"); }            // runs once at load (Extra: static
  constructor(name, amt){
    if (new.target === Account) throw new TypeError("abstract");
    this.name = name;
    this._balance = amt;
    this.#id = ++Account.#aid;
  }
  get id(){ return this.#id; }                     // getter only
  get name(){ return this.#name; }
  set name(v){ if (v.length < 4 || v.length > 15) throw new Error("4 to 15 characters"
  get _balance(){ return this.#balance; }
  set _balance(v){ this.#balance = v; }
  deposit(amt){ this._balance += amt; }
  static payinterest(holder){ holder.deposit(holder._balance * Account.interest); }
  toString(){ return this.name + " " + this._balance; }
}
```

Generators and iterators

Deck 06 · ES_priv_pro_member

yield and function*

A generator function is written `function*` (or `*method()` in a class). `yield` pauses the function and hands a value to the caller, like a `return` that lets the function carry on later from the same place.

```
function* generateSequence(){
  yield 1;
  yield 2;
  yield 3;
}
let sequence = [...generateSequence()];   // spread collects every yielded value
alert(sequence);                           // 1,2,3
```

Generator method in a class

generator method in a class

```
class Polygon {
  constructor(...sides){ this.sides = sides; }
  *getSides(){
    for (const side of this.sides) yield side;
  }
}
const pentagon = new Polygon(1, 2, 3, 4, 5);
[...pentagon.getSides()];    // [1, 2, 3, 4, 5]
```

Using the iterator by hand

```
class MyClass {
  *createIterator(){ yield 1; yield 2; yield 3; }
}
let iterator = new MyClass().createIterator();
iterator.next();   // { value: 1, done: false }
iterator.next();   // { value: 2, done: false }
iterator.next();   // { value: 3, done: false }
iterator.next();   // { value: undefined, done: true }

for (let x of iterator) alert(x);    // a loop works too
```

- The generator object has three methods: `next()`, `return()`, `throw()`.
- Each `next()` runs the function until the next `yield` and returns an object with `value` and `done`. `done` becomes `true` when there is nothing left.
- A generator is a good way to let callers loop over data kept inside an object.

yield* : iterate a whole collection

```
class Employee {
  constructor(nm, id){ this.name = nm; this.id = id; }
  toString(){ return this.name + " " + this.id; }
}
class Company {
  constructor(){
    this.x = new Array();
    this.x[0] = new Employee("Raj", 1);
    this.x[1] = new Employee("Mona", 2);
    this.x[2] = new Employee("Geeta", 3);
  }
  *crIterator(){ yield* this.x; }       // yield* hands over each element of the array
}
var cc = new Company();
for (let e of cc.crIterator()) document.write(e);   // Raj 1, Mona 2, Geeta 3
```

Writing an iterator manually

```
class MyIterator {
  constructor(){ this.i = 0; this.step = new Array(1, 2); }
  [Symbol.iterator](){ return this; }          // makes the object usable in for...of
  next(){
    if (this.i < this.step.length) return { value: this.step[this.i++], done: false };
    return { done: true };
  }
}
let iter = new MyIterator();
iter.next();   // { value: 1, done: false }
iter.next();   // { value: 2, done: false }
iter.next();   // { done: true }
```

CORRECTION TO THE SLIDE

The slide returns the bare element (`return this.step[this.i++]`). A proper iterator's `next()` must return an object with `value` and `done` , as above. The method named `[Symbol.iterator]` is what `for...of` and spread look for.

Last-minute cheat sheet

Both decks

Likely exam questions

- What the `class` keyword gives over constructor functions: non-enumerable methods, strict mode, no hoisting, no call without `new` .
- Write a class with constructor, getter, setter, `toString` and a method.
- Write inheritance with `extends` and `super` ; handle an error with `try / catch` .
- How to make an abstract class (`new.target`).
- What `static` means and how to call it; why the id counter is static.
- Private (`#`) vs protected (`_`) members; why the id has only a getter.
- What the provenance restriction is, with the Base/Derived example.
- How to split classes across files with `export` and `import` and run with Node.
- What a generator is; the result of each `next()` call; `yield` vs `yield*` .
- Difference between `bind` and `call` .

Easy mistakes

- Using the class before it is declared (ReferenceError).
- Naming the getter and the stored field the same, which causes endless recursion. Store in `_name`, expose `name`.
- Using `this` in a child constructor before `super(...)`.
- Reading `#field` outside the class, or using it without declaring it in the class body.
- Thinking `_name` is protected by the language. It is not.
- Calling a static method on an object. Call it on the class: `mymaths.add(3, 5)`.
- Using `this.#privateStatic` in a static method that a subclass may call.
- Calling an unbound method and losing `this`. Use `bind` or call it on the object.
- Forgetting the `*` on a generator method.